

FUNCTIONAL REQUIREMENTS

F1	Available movies: to list all the movies which are currently playing including their language,duration.
F2	List shows: should print all the shows available along with their start time and screen for the movie selected by the customer
F3	Display seats: To display the seat layout for a particular show along with their seat type and AVAILABLE / BOOKED status.
F4	Booking: a customer selects one or more seat numbers for a show. If any selected seat is already BOOKED , the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds .
F5	Pricing: to calculate the billing amount based on the seat type selected by the customer. (SILVER ₹150, GOLD ₹250, PLATINUM ₹400)
F6	Payment: exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED .
F7	Print ticket: to display a ticket based on the seat and movie selected by the customer along with the seat number,total price,booking id,start time and screen.
F8	Cancel booking: this function should cancel the booking made and should make the booked seats again available.

NON FUNCTIONAL REQUIREMENTS

Modularity	To ensure that each class has a single responsibility and is implemented in a separate file.
Extensibility	To ensure that if we add a new class/functionality, the existing classes do not need to be modified.
Input validation	To handle invalid inputs like invalid seat numbers, movies, or payment methods without crashing the entire code.
Reliability	The system should maintain the current seat status during booking and cancellation operations.

NOUN-VERB ANALYSIS

NOUN FOUND	KEEP AS A CLASS?	REASON
Movie	Yes	has its own data and identity
Seat	Yes	has number, type, price
Seat Layout	No	it is a view of a Show's seats, not a thing → a print method
Ticket	No	It is mainly printing/displaying information which can be done by a method
Cinema	Yes	Has its own data and identity
Customer	Yes	Has its own data such as name and phone number
Show	Yes	Represents a screening with a movie, screen and start time
Booking	Yes	Has its own identity/data like booking status, total price or seats.

CORE ENTITY CLASSES

CLASS	DATA MEMBERS	METHODS	MUST NOT DO
MOVIE	Movie name, total duration, language	Constructor, getMovieName, getlanguage, getduration	Must not print tickets, book seats, make payments
SEAT	seatNumber, seatType	Constructor, getSeatNumber, getType	Must not print tickets
SCREEN	Screen number, collection of Seat	Constructor, getScreenNumber	Must not handle payments
CINEMA	cinemaName, collection of movie, collection of show, collection of screen	Constructor, displayCinema	Must not calculate price
SHOW	showId, Movie, screen, collection of ShowSeat	Constructor, getters, displayShow	Must not process payment
SHOWSEAT	Seat, seatstatus	Constructor, releaseseat, book seat, isAvailable	Must not print tickets

CUSTOMER	Name, phone number	Constructor, getters, displayCustomer	Must not manage seats
BOOKING	bookingId, show, totalAmount, status, collection of seats selected	Constructor, getters, confirmBooking, cancelBooking	Must not print tickets

BEHAVIOUR/SERVICE CLASSES

CLASS	DATA MEMBERS	METHODS	MUST NOT DO
Payment (abstract)	No data members	pay(amount)	Must not implement a specific payment method
UpiPayment	upiID	Constructor, pay(amount)	Must not handle booking or ticket printing.
CardPayment	Cardno	Constructor, pay(amount)	Must not handle booking or ticket printing.
CashPayment	No data members	Constructor, pay(amount)	Must not handle booking or ticket printing.
PriceCalculator	Seat type and price	calculatePrice(seats)	Must not book seats or process payment.
TicketPrinter	No data member	printTicket	Must not book seats, calculate price, or process payment.
BookingService	Booking, Show, Payment and PriceCalculator information required for the booking process	bookTicket(), validateSeats(), cancelBooking()	Must not implement individual payment methods or print tickets
Cinema/main menu	Cinema, movies, screens and shows	Main()	Must not contain the business logic of booking, payment or price calculation

Relationships

Pair	Relation	Justification
Cinema — Screen	Composition	Cinema has its own screen .so if a cinema is destroyed screen also gets destroyed.
Screen — Seat	Composition	A screen has its seat, and it belongs to that screen so destroying screen will also delete seat.
Show — Movie	Aggregation	A show has a movie but they both can exist independently
Show — Screen	Aggregation	A show uses a particular screen while the screen can exist independently to host other shows
Show — ShowSeat	Composition	Showseat will display the seats availability for a particular show. So if show is destroyed showseat will also get destroyed.
Booking — Customer	Association	Booking is done through a customer but they can exist independently.
Booking — ShowSeat	Association	Booking uses the showseat to check for the available seats but showseat can exist independently
Booking — Payment	Association	Booking will call a payment method to confirm their booking. If a Booking is destroyed, the Payment object can exist independently
Payment — UpiPayment	Inheritance	upiPayment will inherit the payment class as it is a specialized form of payment.
BookingService — Booking	Association	Booking service uses booking operations but not for a lifetime.

SOLID PRINCIPLES

Principle	Description
S (Single Responsibility)	Booking manages booking data/status, TicketPrinter prints tickets, PriceCalculator calculates price, Show manages show/seats
O (Open -Close)	Add a new payment method by creating a new Payment child class without changing BookingService
L (Liskov Substitution Principle)	Upi, Card, and Cash can all be used through Payment*
I (Interface Segregation Principle)	Don't force every payment type to implement refund() if some don't support refunds
D (Dependency Inversion Principle)	BookingService should receive a Payment*/Payment& instead of creating Card/Upi itself